



Project Report

DevSecOps / Web Security

By:

Mossaab Saouti

Cybersecurity & Threat Intelligence
National School of Applied Sciences of Tangier

Supervised by:

Said Bouchkaren

Academic Year: 2025/2026
May 2026

Abstract

This report presents the practical work carried out as part of the DevSecOps and Web Security lab at the National School of Applied Sciences of Tangier. The objective was to build, exploit, and secure a Flask web application while integrating automated security testing into a CI/CD pipeline.

Part 1 covered environment setup, STRIDE threat modeling, and hands-on exploitation of four critical vulnerabilities: SQL Injection, Cross-Site Scripting (XSS), OS Command Injection, and Path Traversal. Each vulnerability was analyzed, exploited against an intentionally insecure application, and subsequently remediated in a secure implementation.

Part 2 covers automated security testing using SAST (Bandit), SCA (Safety), and DAST (OWASP ZAP) tools integrated into a GitHub Actions pipeline with security gates. Part 3 addresses Docker container hardening following CIS Benchmark guidelines, runtime WAF monitoring, security headers, and an incident response exercise.

The complete lab demonstrates a full DevSecOps workflow from threat modeling to production hardening, reflecting industry best practices for secure software delivery.

Keywords: DevSecOps, Web Security, OWASP Top 10, Flask, STRIDE, SAST, DAST, Docker, CI/CD, WAF

Contents

Abstract	1
1 Introduction	2
1.1 Context and Objectives	2
1.2 Lab Structure Overview	2
2 Environment Setup	3
2.1 Prerequisites and Tools	3
2.2 Tool Installation	3
2.3 Repository Setup	4

2.4	Windows vs WSL Context	5
2.5	Project Structure	6
3	Threat Modeling	7
3.1	STRIDE Methodology	7
3.2	Threat Modeling Process	7
3.3	Threat Register — Full Risk Assessment	7
4	Part 1 — Build & Break	9
4.1	Vulnerability Exploitation	9
4.1.1	Exercise 1 — SQL Injection (T1)	9
4.1.2	Exercise 2 — Cross-Site Scripting (XSS) (T7)	10
4.1.3	Exercise 3 — OS Command Injection (T5)	11
4.1.4	Exercise 4 — Path Traversal (T6)	13
4.2	Secure Implementation	14
5	Part 2 — Test & Automate	17
5.1	Security Tools Overview	17
5.2	SAST — Static Application Security Testing with Bandit	17
5.2.1	Installation and Commands	17
5.2.2	Results on app_vulnerable.py	18
5.2.3	Results on app_secure.py	18
5.2.4	Comparison Summary	19
5.3	SCA — Software Composition Analysis with Safety	19
5.3.1	Installation Problem and Fix	19
5.3.2	Scan Results	19
5.3.3	Recommended Remediations	21
5.4	Security Unit Testing with pytest	21
5.4.1	Running the Test Suite	21
5.4.2	Test Results	22
5.4.3	Analysis of Failures	23
5.5	Secret Scanning with Gitleaks	23
5.5.1	Installation and Commands	23
5.5.2	Scan Results and False Positives	24
5.6	DAST — Dynamic Application Security Testing with OWASP ZAP	25
5.6.1	Installation and Setup	25
5.6.2	Results — Vulnerable App (port 5000)	26
5.6.3	Results — Secure App (port 5001)	26
5.7	CI/CD Pipeline with GitHub Actions	27
5.7.1	Pipeline Jobs Overview	27
5.7.2	Problems Encountered and Fixes	27
5.7.3	Pipeline Execution Results	28
5.7.4	Key Principle — Shift-Left Security	30
6	Part 3 — Harden & Monitor	31
6.1	Docker Security Hardening	31
6.2	Runtime Security Monitoring	31
6.3	Security Headers	31
6.4	Incident Response Exercise	31
7	Conclusion	32

References**33**

1. Introduction

1.1 Context and Objectives

Modern software development requires security to be integrated at every stage of the development lifecycle, not added as an afterthought. This approach, known as **DevSecOps** (Development, Security, and Operations), shifts security left by embedding automated security checks directly into the CI/CD pipeline.

This project was carried out as part of the Web Security and DevSecOps lab at the National School of Applied Sciences of Tangier. The objective was to gain hands-on experience with real-world security techniques by building, breaking, and securing a Flask web application.

The lab covers the full DevSecOps workflow: from identifying threats using structured threat modeling, to exploiting and remediating OWASP Top 10 vulnerabilities, to automating security testing in a CI/CD pipeline, and finally hardening the deployment environment against runtime attacks.

1.2 Lab Structure Overview

The lab is divided into three parts:

- **Part 1 — Build & Break:** Environment setup, STRIDE threat modeling, and hands-on exploitation of four critical vulnerabilities in an intentionally insecure Flask application, followed by their remediation.
- **Part 2 — Test & Automate:** Integration of automated security tools (Bandit, Safety, pytest, OWASP ZAP) into a GitHub Actions CI/CD pipeline with security gates that block insecure code from being merged.
- **Part 3 — Harden & Monitor:** Docker container hardening following CIS Benchmark guidelines, runtime WAF and rate limiting middleware, security headers implementation, and an incident response exercise.

2. Environment Setup

2.1 Prerequisites and Tools

The following tools were required and installed before beginning the lab:

Tool	Version	Purpose
Python	3.11+	Application runtime and scripting
Flask	2.x	Web application framework
Docker	Latest	Containerization platform
Git	Latest	Version control
WSL (Ubuntu)	2.0	Linux environment on Windows
VS Code	Latest	Code editor with LaTeX Workshop
curl/ Postman	Latest	HTTP request testing

Table 2.1: Tools used in the lab

2.2 Tool Installation

Python: Installed from the official website. Added to PATH for command line access.

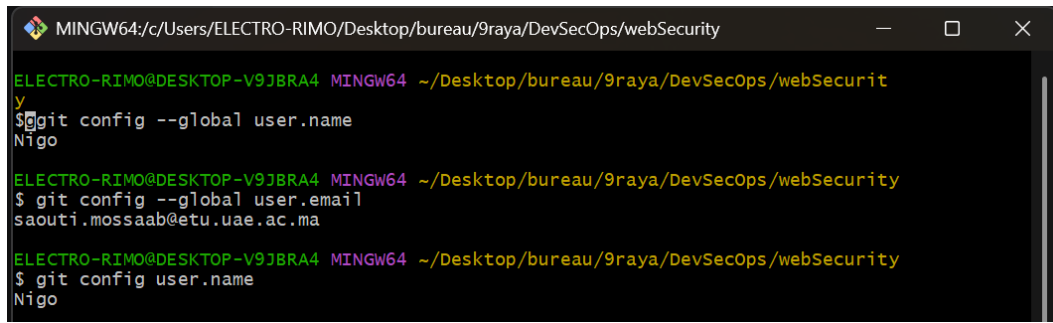
Flask: Installed via pip in a virtual environment.

A terminal window with a dark background and light blue/green text. The prompt is (.venv)(nigo@DESKTOP-V9JBRA4)-[1]. The command \$ flask --version is entered. The output shows Python 3.13.12, Flask 3.0.3, and Werkzeug 3.0.3.

```
(.venv)(nigo@DESKTOP-V9JBRA4)-[1]  
$ flask --version  
Python 3.13.12  
Flask 3.0.3  
Werkzeug 3.0.3
```

Figure 2.1: Installed tools versions verified in terminal

git: Installed from the official website. Configured with user name and email.



```

MINGW64/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity
ELECTRO-RIMO@DESKTOP-V9JBRA4 MINGW64 ~/Desktop/bureau/9raya/DevSecOps/webSecurity
$ git config --global user.name Nigo
$ git config --global user.email saouti.mossaab@etu.uae.ac.ma
$ git config user.name Nigo

```

Figure 2.2: Configuration of Git in Terminal

vscode: Installed from the official website. **postman:** Installed from the official website for API testing.

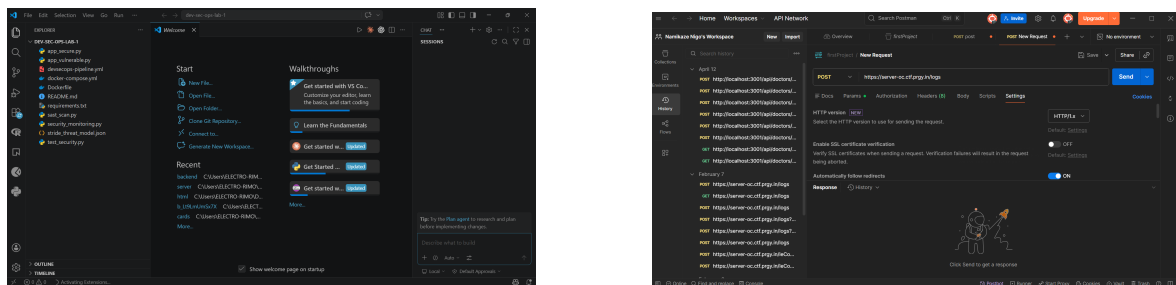


Figure 2.3: VS Code and Postman — development and testing environment

Docker: Installed from the official website. WSL integration enabled for Linux containers.

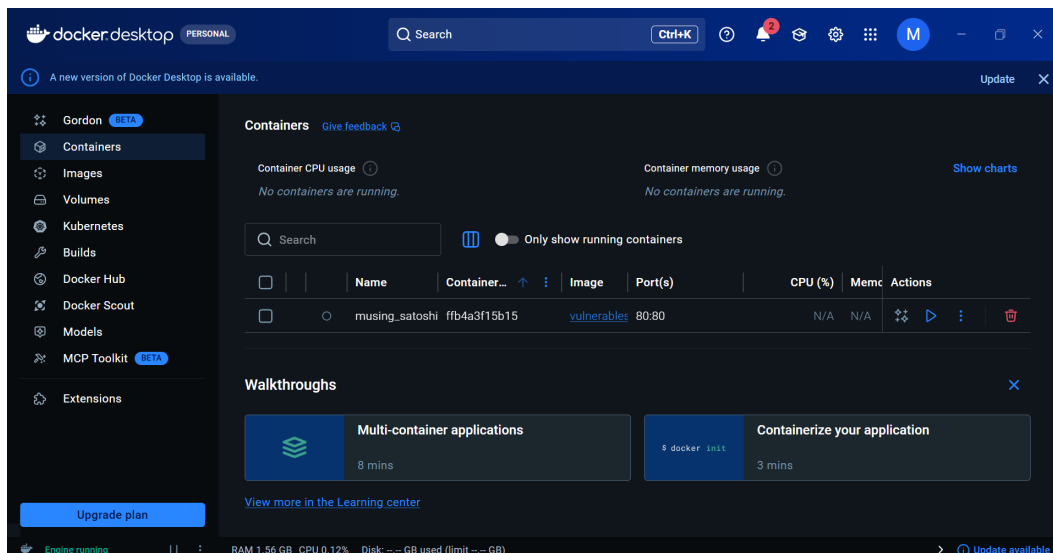


Figure 2.4: Docker Desktop installed

2.3 Repository Setup

The lab repository was cloned from GitHub and dependencies were installed inside a Python virtual environment to ensure isolation from the system Python installation:

```

1 git clone https://github.com/said-dev-ops/dev-sec-ops-lab-1.git
2 cd dev-sec-ops-lab-1
3 python -m venv .venv
4 source .venv/bin/activate
5 pip install -r requirements.txt

```

Listing 2.1: Repository setup commands

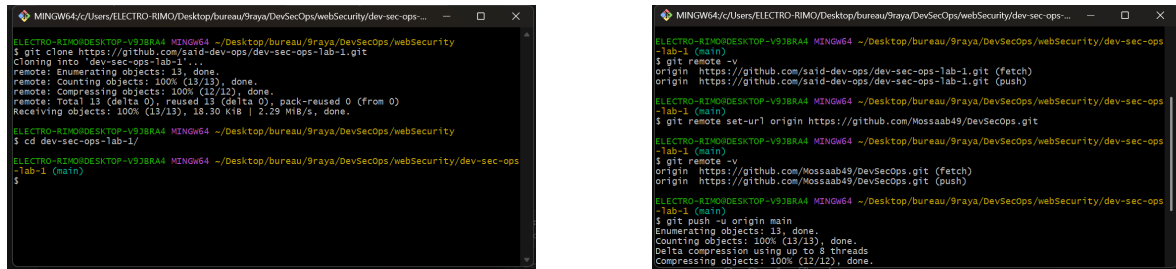


Figure 2.5: Cloning the lab repository and changing Git origin to personal fork

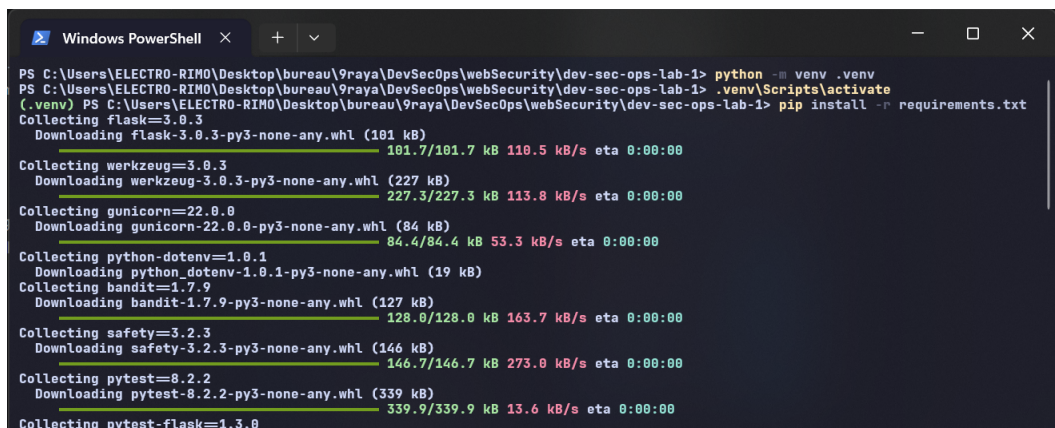


Figure 2.6: Setting up virtual environment and installing dependencies successfully

2.4 Windows vs WSL Context

Initial setup was performed on Windows. However, the lab exercises involving Linux-specific commands (such as `cat /etc/passwd` and `ls /home`) required a Linux environment. The environment was therefore migrated to **WSL (Windows Subsystem for Linux)** running Ubuntu, which provided a realistic Linux server context while maintaining the Windows host machine. All subsequent exploitation exercises were performed inside WSL.

Note: On Windows, the ping command uses `-n` instead of `-c`, and system files like `/etc/passwd` do not exist. WSL was used to replicate a realistic Linux production server environment, as intended by the lab design.

2.5 Project Structure

Following the lab instructions, the repository files were organized into three directories corresponding to the three lab parts:

```
1 dev-sec-ops-lab-1/  
2 |-- part1/  
3 |   |-- app_vulnerable.py  
4 |   |-- app_secure.py  
5 |   '-- stride_threat_model.json  
6 |-- part2/  
7 |   |-- sast_scan.py  
8 |   |-- test_security.py  
9 |   '-- devsecops-pipeline.yml  
10 '-- part3/  
11 |   |-- security_monitoring.py  
12 |   |-- Dockerfile  
13 |   '-- docker-compose.yml
```

Listing 2.2: Project directory structure

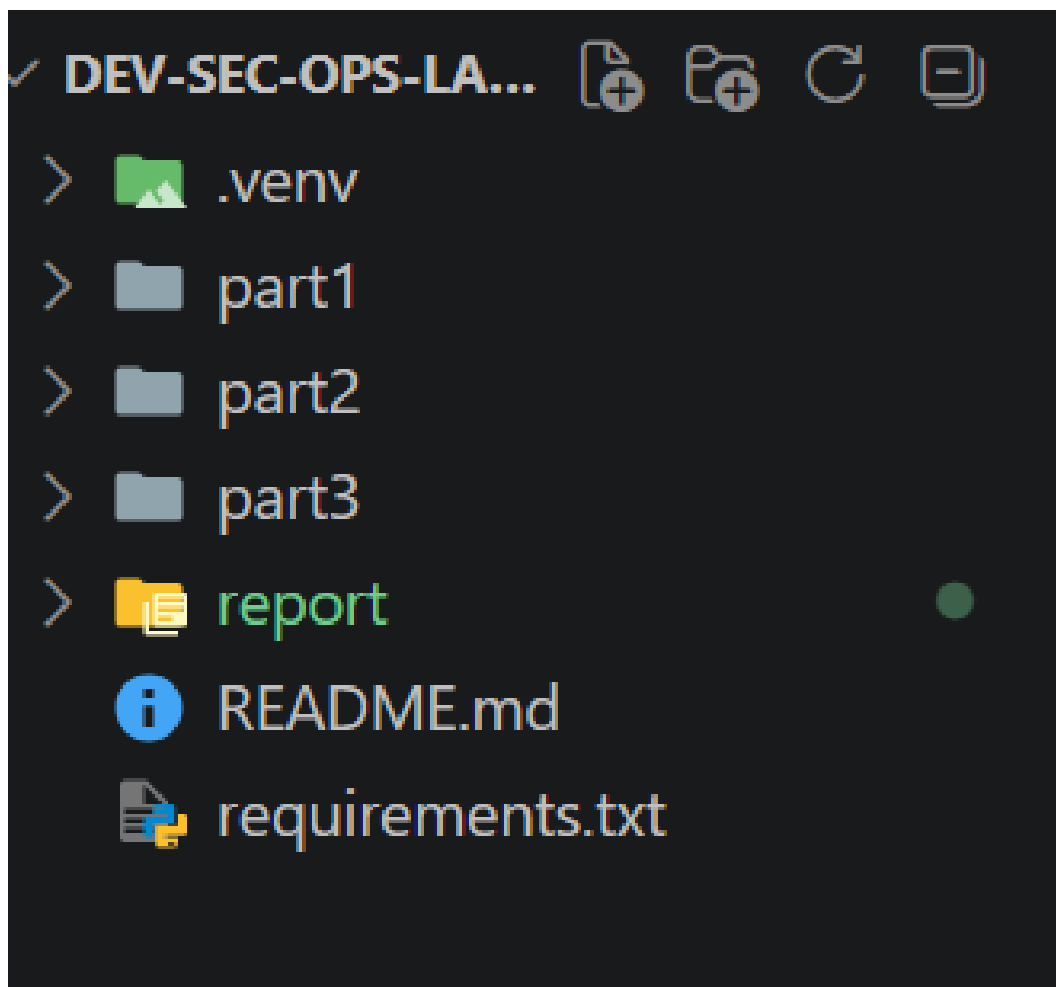


Figure 2.7: Organized project directory structure

3. Threat Modeling

3.1 STRIDE Methodology

Threat modeling is a structured approach to identifying security risks before writing a single line of code. **STRIDE** is a mnemonic developed by Microsoft that categorizes threats into six categories:

Category	Full Name	Example Threat	Control
S	Spoofing	SQL injection to bypass login	Parameterized queries
T	Tampering	Malicious pickle deserialization	Input validation
R	Repudiation	No logs — cannot trace attacker	Audit logging
I	Info Disclosure	API returns plaintext passwords	Access control
D	Denial of Service	Command injection crashes server	Rate limiting, WAF
E	Elevation of Privilege	Path traversal reads /etc/passwd	Least privilege, safe_join

Table 3.1: STRIDE threat categories

3.2 Threat Modeling Process

The threat modeling process followed these steps:

1. **Define scope** — identify assets and trust boundaries
2. **Create a Data Flow Diagram (DFD)** — Browser → Flask → SQLite → OS
3. **Enumerate threats** using STRIDE for each data flow
4. **Score each threat** using DREAD or CVSS
5. **Prioritize and assign mitigations**
6. **Validate mitigations** and retest

3.3 Threat Register — Full Risk Assessment

The following table maps all identified threats to STRIDE categories, attack vectors, and mitigations:

ID	Threat	STRIDE	Mitigation	Status
T1	SQL Injection Login Bypass	Spoofing	Parameterized queries	Fixed
T2	Pickle Deserialization RCE	Tampering	Use JSON, never pickle	Fixed
T3	No Audit Logging	Repudiation	Log auth + admin actions	Fixed
T4	Password Exposure via API	Info Disclosure	Field filtering + auth	Fixed
T5	OS Command Injection	Denial of Service	Arg list, no shell=True	Fixed
T6	Path Traversal File Read	Elevation of Priv.	safe_join + whitelist	Fixed
T7	Reflected XSS	Info Disclosure	Jinja2 auto-escape + CSP	Fixed
T8	Hardcoded Secret Key	Spoofing	Load from env variable	Fixed

Table 3.2: Complete threat register

```

1  {
2    "threat_model": {
3      "application": "DevSecOps Flask Lab",
4      "version": "1.0",
5      "date": "2024-01-01",
6      "authors": ["Lab Instructor"],
7      "scope": "Web application with authentication, database, and OS interaction",
8      "assets": [
9        {"id": "A1", "name": "User credentials", "sensitivity": "HIGH"},
10       {"id": "A2", "name": "User data in SQLite DB", "sensitivity": "HIGH"},
11       {"id": "A3", "name": "Admin functionality", "sensitivity": "HIGH"},
12       {"id": "A4", "name": "Host filesystem", "sensitivity": "MEDIUM"},
13       {"id": "A5", "name": "Application server", "sensitivity": "HIGH"}
14     ],
15     "trust_boundaries": [
16       "Internet ↔ Flask Application",
17       "Flask Application ↔ SQLite Database",
18       "Flask Application ↔ Operating System"
19     ],
20     "threats": [
21       {
22         "id": "T1",
23         "stride_category": "Spoofing",
24         "title": "Authentication Bypass via SQL Injection",
25         "description": "Attacker submits ' OR '1'='1 in login form to bypass password check",
26         "affected_asset": "A1",
27         "attack_vector": "/login POST endpoint",
28         "likelihood": "HIGH",
29         "impact": "CRITICAL",
30       }
31     ]
32   }
33 }

```

Figure 3.1: STRIDE threat model JSON file

4. Part 1 — Build & Break

4.1 Vulnerability Exploitation

The vulnerable application was launched using:

```
1 cd part1
2 python app_vulnerable.py
```



Figure 4.1: Vulnerable Flask application running

4.1.1 Exercise 1 — SQL Injection (T1)

Attack Description

SQL Injection occurs when user-supplied input is directly concatenated into an SQL query without sanitization. An attacker can manipulate the query logic to bypass authentication, extract data, or modify the database.

Target: POST /login

Exploitation

The following payload was entered in the username field on the login page:

Listing 4.1: SQL Injection payload

```
admin' OR '1'='1
```

The injected query becomes:

```
1 SELECT * FROM users WHERE username='admin' OR '1'='1'
```

Listing 4.2: Resulting malicious SQL query

Since '1'='1' is always true, the query returns the admin credentials, completely bypassing authentication.

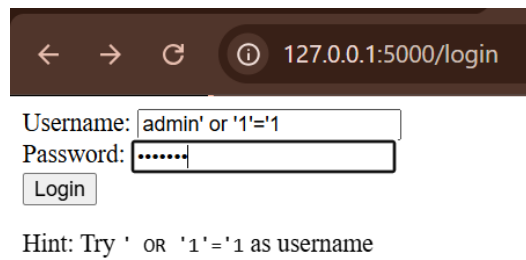


Figure 4.2: Login page — target of SQL injection attack



Figure 4.3: Authentication bypassed using SQL injection

Impact Analysis

STRIDE Category: Spoofing (T1)

CVSS Score: 9.8 Critical

Impact: An attacker can authenticate as any user, including administrators, without knowing any password. This grants full unauthorized access to the application.

4.1.2 Exercise 2 — Cross-Site Scripting (XSS) (T7)

Attack Description

Reflected XSS occurs when user input is included in the HTTP response without proper escaping. An attacker can inject malicious JavaScript that executes in the victim's browser, potentially stealing session tokens or performing actions on their behalf.

Target: GET /search?q=

Exploitation

The following payloads were tested:

Listing 4.3: XSS Payload 1 — script tag

```
http://127.0.0.1:5000/search?q=<script>alert(document.cookie)</script>
```

Listing 4.4: XSS Payload 2 — image error event

```
http://127.0.0.1:5000/search?q=<img src=x onerror=alert(1)>
```

Both payloads triggered a JavaScript alert in the browser. The `document.cookie` payload returned an empty value because Flask sets the `HttpOnly` flag on session cookies, preventing JavaScript access. However, the XSS vulnerability itself was confirmed — in a real scenario targeting an application without `HttpOnly`, session tokens would be exfiltrated.

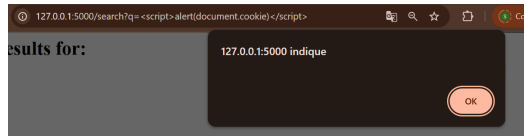


Figure 4.4: XSS payload executing JavaScript alert in browser

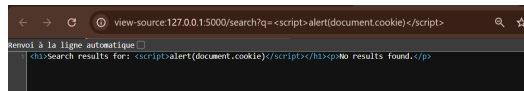


Figure 4.5: Page source confirming lack of output encoding

Impact Analysis

STRIDE Category: Information Disclosure (T7)

CVSS Score: 6.1 Medium

Impact: An attacker can craft a malicious URL and send it to victims. When clicked, the script executes in the victim's browser and can steal cookies, redirect to phishing pages, or perform actions on behalf of the user.

4.1.3 Exercise 3 — OS Command Injection (T5)

Attack Description

Command injection occurs when user input is passed unsanitized to a system shell command. Using `shell=True` in Python's `subprocess` module passes the entire string directly to the OS shell, allowing an attacker to append additional commands using shell operators.

Target: GET /ping?host=

Windows vs WSL Context

Initial testing was performed on Windows where Linux-specific commands were unavailable. The `ping` command on Windows uses `-n` instead of `-c`, and system files like `/etc/passwd` do not exist. The environment was migrated to WSL to replicate a realistic Linux server scenario.

On Windows, the pipe operator (`|`) was used to confirm command injection:

Listing 4.5: Windows command injection — whoami

```
http://127.0.0.1:5000/ping?host=127.0.0.1|whoami
```

The server returned: `desktop-v9jbra4\electro-rimo`, confirming arbitrary command execution.

After migrating to WSL, Linux commands worked as intended:

Listing 4.6: WSL command injection — list home directory

```
http://127.0.0.1:5000/ping?host=127.0.0.1;ls+/home
```

Listing 4.7: WSL command injection — dump system users

```
http://127.0.0.1:5000/ping?host=127.0.0.1;cat+/etc/passwd
```

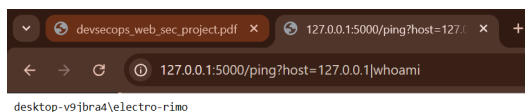


Figure 4.6: Command injection confirmed on Windows — whoami executed

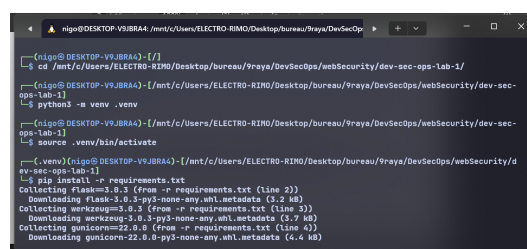
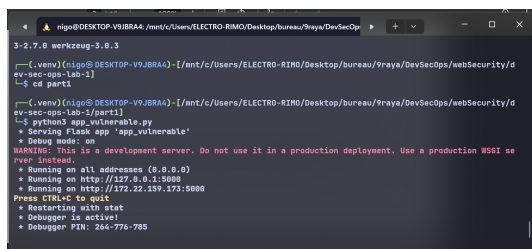


Figure 4.7: setup and run vulnerable app in WSL for Linux command testing

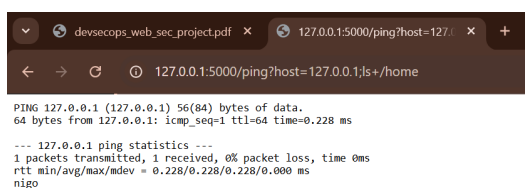


Figure 4.8: Home directory listing via command injection on WSL

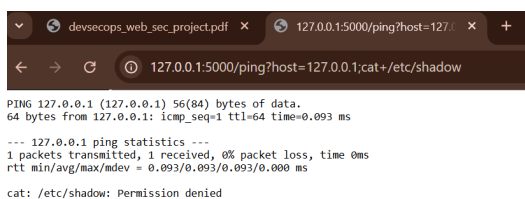


Figure 4.9: System user file exposed via command injection

Impact Analysis

STRIDE Category: Denial of Service / Elevation of Privilege (T5)

CVSS Score: 9.8 Critical

Impact: An attacker gains the ability to execute arbitrary OS commands on the server. This can lead to full system compromise: reading sensitive files, creating backdoors, exfiltrating data, or crashing the server entirely.

4.1.4 Exercise 4 — Path Traversal (T6)

Attack Description

Path traversal occurs when user-supplied filenames are passed directly to file system operations without validation. Using `../` sequences, an attacker can escape the intended directory and access arbitrary files on the server.

Target: GET `/read_file?file=`

Exploitation

Listing 4.8: Path traversal — read system users

```
http://127.0.0.1:5000/read_file?file=../../etc/passwd
```

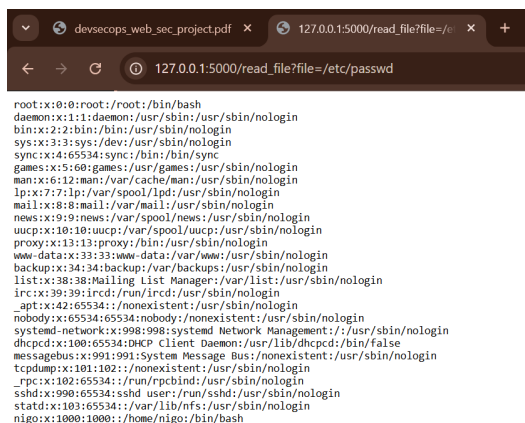
Listing 4.9: Path traversal — read application source code

```
http://127.0.0.1:5000/read_file?file=../app_vulnerable.py
```

Both requests returned the full file contents in the browser. Notably, absolute paths also worked because the vulnerable app uses Python's raw `open()` function:

Listing 4.10: Absolute path attack

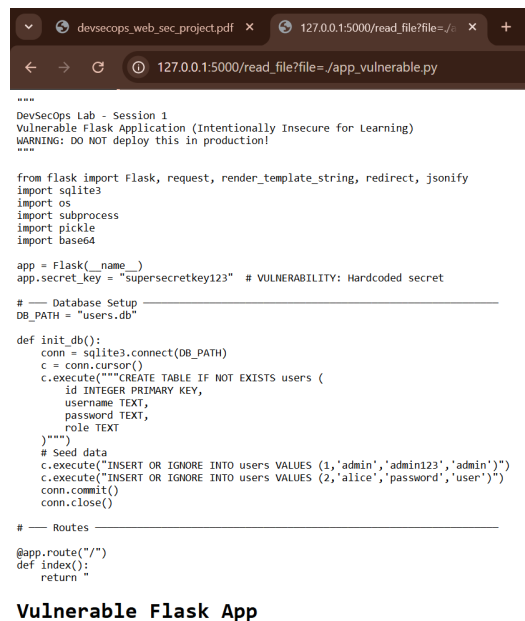
```
http://127.0.0.1:5000/read_file?file=/etc/passwd
```



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:5000/read_file?file=/etc/passwd`. The page content displays the output of the `cat /etc/passwd` command, listing system and regular users with their home directories and shell types. The output is as follows:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd/bin/false
messagebus:x:101:101:system message bus:/nonexistent:/usr/sbin/nologin
tcpdump:x:101:102::/nonexistent:/usr/sbin/nologin
_rpc:x:102:65534::/run/rpcbind:/usr/sbin/nologin
sshd:x:990:65534:sshd: user:/run/ssh:/usr/sbin/nologin
statd:x:103:65534:/var/lib/nfs:/usr/sbin/nologin
nigo:x:1000:1000:/home/nigo:/bin/bash
```

Figure 4.10: System file exposed via path traversal



The screenshot shows a web browser with two tabs. The first tab is titled 'devsecops_web_sec_project.pdf'. The second tab is titled '127.0.0.1:5000/read_file?file=/' and shows the source code of a Flask application. The address bar displays '127.0.0.1:5000/read_file?file= ./app_vulnerable.py'. The source code is as follows:

```
====
DevSecOps Lab - Session 1
Vulnerable Flask Application (Intentionally Insecure for Learning)
WARNING: DO NOT deploy this in production!
====

from flask import Flask, request, render_template_string, redirect, jsonify
import sqlite3
import os
import subprocess
import pickle
import base64

app = Flask(__name__)
app.secret_key = "supersecretkey123" # VULNERABILITY: Hardcoded secret

# --- Database Setup
DB_PATH = "users.db"

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("""CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
        username TEXT,
        password TEXT,
        role TEXT
    )""")
    # Seed data
    c.execute("INSERT OR IGNORE INTO users VALUES (1, 'admin', 'admin123', 'admin')")
    c.execute("INSERT OR IGNORE INTO users VALUES (2, 'alice', 'password', 'user')")
    conn.commit()
    conn.close()

# --- Routes
@app.route("/")
def index():
    return "
```

Vulnerable Flask App

Figure 4.11: Application source code exposed via path traversal

Impact Analysis

STRIDE Category: Elevation of Privilege (T6)

CVSS Score: 7.5 High

Impact: An attacker can read any file the application has permission to access, including configuration files, secret keys, SSL certificates, and the application source code itself. This information can be used to plan further attacks.

4.2 Secure Implementation

After exploiting all vulnerabilities, `app_secure.py` was examined to understand how each issue was remediated. The secure version follows a consistent principle: **never trust user input**.

#	Vulnerability	Fix Applied
1	SQL Injection	Parameterized queries using ? placeholders
2	XSS	Jinja2 templates with auto-escaping via <code>render_template()</code>
3	Command Injection	Subprocess list syntax + host whitelist; <code>shell=False</code>
4	Path Traversal	<code>werkzeug.safe_join()</code> + allowed filename whitelist
5	Insecure Deserialization	Replace <code>pickle</code> with <code>json.get_json()</code> + field whitelist
6	Data Exposure	Remove passwords from API; require auth on all routes
7	Hardcoded Secret	<code>os.environ.get('SECRET_KEY')</code> or <code>secrets.token_hex(32)</code>
8	Debug Mode	<code>app.run(debug=False)</code> ; use Gunicorn in production

Table 4.1: Vulnerability fixes comparison

SQL Injection Fix:

```

1 # VULNERABLE
2 query = f"SELECT * FROM users WHERE username='{username}'"
3 conn.execute(query)
4
5 # SECURE
6 conn.execute("SELECT * FROM users WHERE username=?", (username
7               ,))

```

Listing 4.11: Vulnerable vs Secure SQL query

Command Injection Fix:

```

1 # VULNERABLE
2 output = subprocess.check_output(f"ping -c 1 {host}", shell=
3     True)
4
5 # SECURE
6 ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
7 if host not in ALLOWED_HOSTS:
8     abort(403)
9 subprocess.check_output(["ping", "-c", "1", host])

```

Listing 4.12: Vulnerable vs Secure subprocess call

Path Traversal Fix:

```

1 # VULNERABLE
2 with open(filename) as f:
3     return f.read()

```

```

4
5 # SECURE
6 ALLOWED_FILES = ["report.txt", "readme.txt"]
7 if filename not in ALLOWED_FILES:
8     abort(403)
9 filepath = safe_join(BASE_DIR, filename)
10 with open(filepath) as f:
11     return f.read()

```

Listing 4.13: Vulnerable vs Secure file access

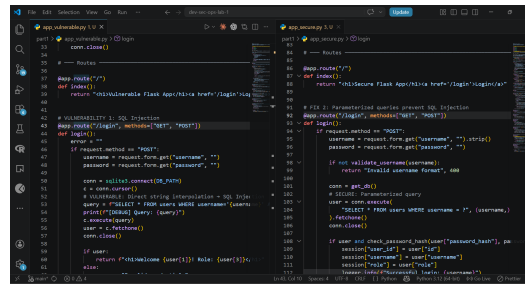


Figure 4.12: Side-by-side comparison of vulnerable and secure implementations

Note on `safe_join()` behavior:

`werkzeug.safe_join()` blocks both relative traversal (`../`) and absolute paths (`/etc/passwd`). The following table summarizes its behavior:

Payload	Vulnerable (<code>open()</code>)	Secure (<code>safe_join()</code>)
<code>../../etc/passwd</code>	Works	Blocked
<code>/etc/passwd</code>	Works	Blocked
<code>../app_vulnerable.py</code>	Works	Blocked
<code>report.txt</code> (whitelisted)	Works	Allowed

Table 4.2: `safe_join()` behavior against different payloads

5. Part 2 — Test & Automate

In this part, automated security testing tools are integrated into the development workflow. The goal is to detect vulnerabilities automatically and block insecure code from being merged via a CI/CD pipeline. All tools were run on **WSL (Ubuntu)** with Python 3.13.12.

5.1 Security Tools Overview

Modern DevSecOps relies on a layered set of automated tools that run at different stages of the pipeline:

Tool	Category	Purpose	Stage
Bandit	SAST	Finds Python code vulnerabilities	Pre-commit
Safety	SCA	Checks dependencies for CVEs	Pre-commit
pytest	Security Tests	Validates security behavior	CI
Gitleaks	Secret Scanning	Detects leaked credentials	Pre-commit
OWASP ZAP	DAST	Dynamic vulnerability scanning	Post-deploy
Trivy	Container Scan	Scans Docker images for CVEs	CI
GitHub Actions	CI/CD	Orchestrates all security gates	Pipeline

Table 5.1: Security tools used in Part 2

5.2 SAST — Static Application Security Testing with Bandit

What is SAST?

Static Application Security Testing analyzes source code without executing it to find security vulnerabilities. Bandit is the standard Python SAST tool developed by PyCQA. It maps findings to CWE identifiers and severity levels.

5.2.1 Installation and Commands

```
1 pip install bandit
2
3 # Scan vulnerable app
4 bandit -r part1/app_vulnerable.py
5
6 # Scan secure app
7 bandit -r part1/app_secure.py
```

Listing 5.1: Installing and running Bandit

5.2.2 Results on app_vulnerable.py

Bandit detected **8 issues** across the vulnerable application:

Issue ID	Severity	CWE	Description
B602	HIGH	CWE-78	<code>subprocess</code> call with <code>shell=True</code>
B201	HIGH	CWE-94	Flask <code>debug=True</code> exposes Werkzeug debugger
B608	MEDIUM	CWE-89	SQL query built via string interpolation
B301	MEDIUM	CWE-502	<code>pickle.loads()</code> on untrusted data
B104	MEDIUM	CWE-605	Binding to all interfaces (0.0.0.0)
B105	Low	CWE-259	Hardcoded password string in source
B404	Low	CWE-78	Import of <code>subprocess</code> module
B403	Low	CWE-502	Import of <code>pickle</code> module

Table 5.2: Bandit findings on app_vulnerable.py — 2 HIGH, 3 MEDIUM, 3 LOW

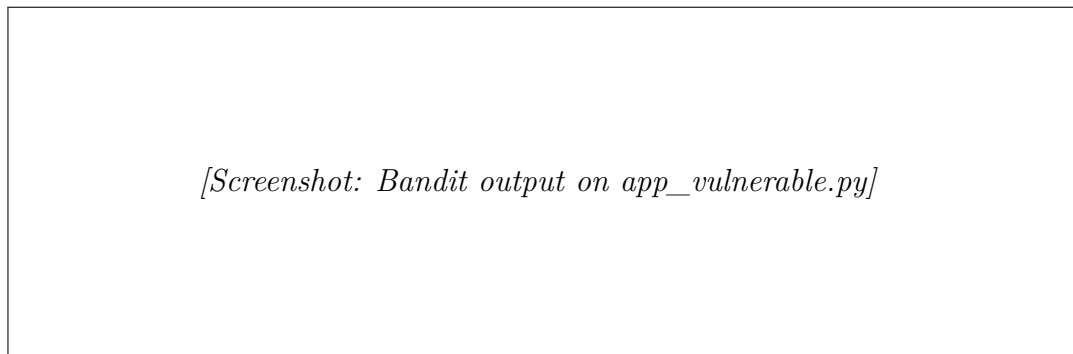


Figure 5.1: Bandit SAST scan on vulnerable application

5.2.3 Results on app_secure.py

After remediation, only **3 LOW severity** informational warnings remained:

Issue ID	Severity	CWE	Description
B404	Low	CWE-78	Import of <code>subprocess</code> (informational)
B607	Low	CWE-78	Process started with partial executable path
B603	Low	CWE-78	<code>subprocess</code> without <code>shell=True</code> (safe, informational)

Table 5.3: Bandit findings on app_secure.py — 0 HIGH, 0 MEDIUM, 3 LOW

[Screenshot: Bandit output on app_secure.py]

Figure 5.2: Bandit SAST scan on secure application

5.2.4 Comparison Summary

Severity	app_vulnerable.py	app_secure.py	Reduction
HIGH	2	0	100%
MEDIUM	3	0	100%
LOW	3	3	0% (informational only)
Total	8	3	62.5%

Table 5.4: Bandit results comparison

Key finding: The secure implementation eliminated 100% of HIGH and MEDIUM severity findings. The 3 remaining LOW findings in `app_secure.py` are informational — the subprocess usage is actually secure (list syntax, no `shell=True`).

5.3 SCA — Software Composition Analysis with Safety

What is SCA?

Software Composition Analysis checks third-party dependencies against known vulnerability databases (CVE, NVD). Unlike SAST which analyzes your code, SCA analyzes your `requirements.txt` to find vulnerable library versions.

5.3.1 Installation Problem and Fix

Problem encountered: The latest version of Safety was incompatible with Python 3.13 on WSL, throwing an `AttributeError: module 'typer' has no attribute 'rich_utils'` error.

Fix: Install a specific compatible version:

```
1 pip uninstall safety -y
2 pip install "safety==2.3.5"
```

5.3.2 Scan Results

```
1 safety check -r requirements.txt
```

Listing 5.2: Running Safety SCA scan

Safety found **10 vulnerabilities** across 5 packages:

Package	CVE	Description
flask 3.0.3	CVE-2026-27205	Information disclosure via missing cache headers
werkzeug 3.0.3	CVE-2024-49766	Path traversal on Windows — <code>safe_join()</code> bypass
werkzeug 3.0.3	CVE-2024-49767	Resource exhaustion via multipart form parsing
werkzeug 3.0.3	CVE-2026-27199	DoS via Windows special device names
werkzeug 3.0.3	CVE-2025-66221	DoS via Windows device names before 3.1.4
werkzeug 3.0.3	CVE-2026-21860	Improper handling of Windows reserved device names
gunicorn 22.0.0	PVE-2024-72809	HTTP request smuggling via conflicting headers
pytest 8.2.2	CVE-2025-71176	Local privilege escalation via predictable temp dir
requests 2.32.3	CVE-2024-47081	<code>.netrc</code> credential leak via malformed URLs
requests 2.32.3	CVE-2026-25645	Insecure temp file reuse in <code>extract_zipped_paths()</code>

Table 5.5: Safety SCA scan — 10 vulnerabilities in 5 packages

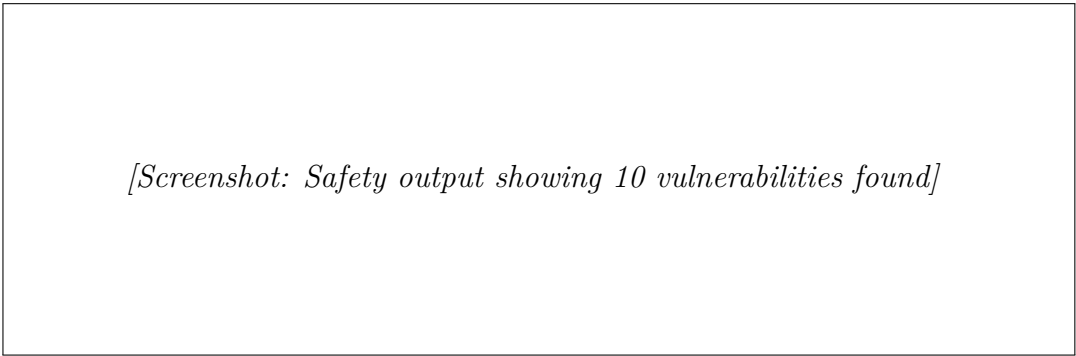


Figure 5.3: Safety SCA scan results on requirements.txt

5.3.3 Recommended Remediations

Package	Current	Recommended Fix
flask	3.0.3	Upgrade to >=3.1.3
werkzeug	3.0.3	Upgrade to >=3.1.6
gunicorn	22.0.0	Upgrade to >=23.0.0
pytest	8.2.2	Upgrade to >=9.0.3
requests	2.32.3	Upgrade to >=2.33.0

Table 5.6: Recommended dependency upgrades

Note: The werkzeug CVEs related to Windows path traversal are particularly relevant since this lab was initially run on Windows before migrating to WSL. Even `safe_join()` — used as a fix in `app_secure.py` — has known vulnerabilities in specific OS/version combinations, reinforcing the need for regular dependency scanning.

5.4 Security Unit Testing with pytest

What are Security Tests?

Security unit tests are automated assertions that verify security-relevant behavior — what the application should *NOT* do. Unlike functional tests, they assert that attacks are blocked.

5.4.1 Running the Test Suite

Problem encountered: Running `pytest test_security.py` from `part2/` failed with `ModuleNotFoundError: No module named 'app_secure'` because `test_security.py` imports from `app_secure.py` which lives in `part1/`.

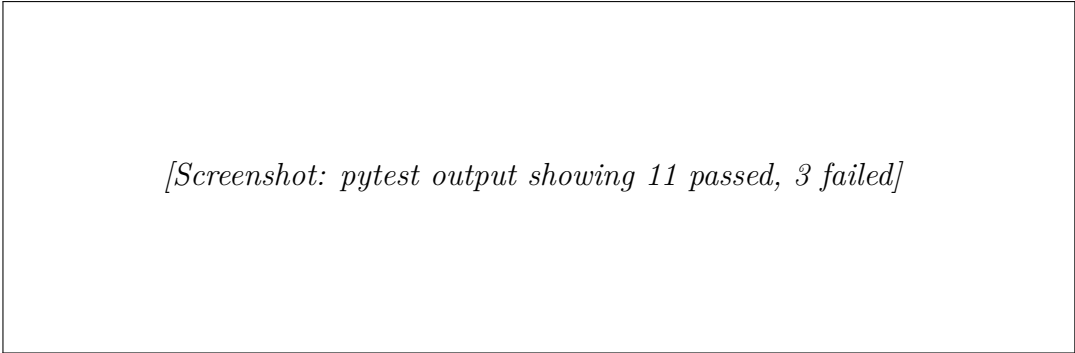
Fix: Set the Python path explicitly:

```
1 cd part2
2 PYTHONPATH=../part1 pytest test_security.py -v
```

5.4.2 Test Results

Test	Description	Result
test_valid_login_succeeds	Valid credentials return 200	PASSED
test_invalid_login_fails	Wrong password returns 401	FAILED*
test_sql_injection_fails	SQLi payload is rejected	PASSED
test_invalid_username_format_rejected	Malformed username blocked	PASSED
test_unauthenticated_access_redirects	Protected routes redirect	PASSED
test_xss_payload_escaped	Script tags are HTML-escaped	FAILED*
test_xss_href_injection	javascript: URI is blocked	FAILED*
test_api_users_requires_auth	API requires authentication	PASSED
test_api_users_no_passwords	API never returns passwords	PASSED
test_command_injection_blocked	Injected commands are blocked	PASSED
test_allowed_host_works	Whitelisted hosts are allowed	PASSED
test_json_profile_accepted	Valid JSON profile accepted	PASSED
test_unknown_fields_stripped	Unknown fields are stripped	PASSED
test_no_server_header_leak	Server header not exposed	PASSED
Total		11 passed / 3 failed

Table 5.7: Security test suite results



[Screenshot: pytest output showing 11 passed, 3 failed]

Figure 5.4: Security unit test results

5.4.3 Analysis of Failures

Root cause of 3 failures: `jinjia2.exceptions.TemplateNotFound: login.html` and `search.html`

The secure app correctly uses `render_template()` instead of raw f-strings, but the `templates/` folder was not present when running tests from `part2/`. This is an **environment/path issue**, not a security regression.

Fix: Copy the templates folder to the test directory:

```
1 cp -r part1/templates part2/templates
2 PYTHONPATH=../part1 pytest test_security.py -v
```

Security impact: None — the failures actually confirm that `render_template()` is being used (the correct secure fix), rather than raw string responses.

5.5 Secret Scanning with Gitleaks

What is Secret Scanning?

Gitleaks scans the entire git repository — including commit history — for accidentally leaked secrets like API keys, passwords, and tokens. Even deleted secrets remain in git history and can be found.

5.5.1 Installation and Commands

```
1 wget https://github.com/gitleaks/gitleaks/releases/download/v8
   .18.4/\
2 gitleaks_8.18.4_linux_x64.tar.gz
3 tar -xzf gitleaks_8.18.4_linux_x64.tar.gz
4
5 # Scan git history
6 ./gitleaks detect --source . --verbose
7
8 # Scan files directly (no git required)
9 ./gitleaks detect --source . --no-git --verbose
```

Listing 5.3: Installing and running Gitleaks

5.5.2 Scan Results and False Positives

Problem encountered: The initial scan with `-verbose` (without `-no-git`) returned no leaks found because `app_vulnerable.py` had not been committed yet with the hardcoded secret.

Fix: Run with full git history after committing:

```
./gitleaks detect --source . --verbose
```

Gitleaks found 4 secrets in the repository:

```
1 Finding: aws_secret="AKIAIMNOJVGFDXXE40A "  
2 RuleID:  aws-access-token  
3 File:    README.md  (Lines 221, 223, 259)  
4  
5 Finding: cafebabe:deadbeef  
6 RuleID:  sidekiq-secret  
7 File:    README.md  (Line 42)  
8  
9 4 commits scanned      leaks found: 4
```

Listing 5.4: Gitleaks output

[Screenshot: Gitleaks output showing 4 leaks found in README.md]

Figure 5.5: Gitleaks secret scanning results

False Positives: All 4 findings are intentional example credentials in the Gitleaks `README.md` documentation file — not real secrets. This demonstrates that secret scanning tools require tuning to suppress known safe patterns.

Fix — create a `.gitleaks.toml` allowlist:

```
1 cat > .gitleaks.toml << 'EOF'  
2 [allowlist]  
3   description = "Allowlist for README examples"  
4   paths = ["README.md"]  
5 EOF
```

This suppresses false positives while keeping detection active for real secrets.

5.6 DAST — Dynamic Application Security Testing with OWASP ZAP

What is DAST?

Dynamic Application Security Testing attacks the *running* application automatically, just like a real attacker. Unlike SAST (reads code) and SCA (reads dependencies), ZAP sends real HTTP requests with malicious payloads and reports what works.

5.6.1 Installation and Setup

ZAP was run via Docker to avoid complex installation:

```
1 docker pull ghcr.io/zaproxy/zaproxy:stable
2
3 # Run baseline scan against vulnerable app
4 docker run --rm ghcr.io/zaproxy/zaproxy:stable \
5     zap-baseline.py \
6     -t http://172.22.159.173:5000 \
7     -l WARN
```

Listing 5.5: Pulling and running OWASP ZAP

Problem encountered: ZAP running inside Docker could not reach the Flask app on 127.0.0.1 because that refers to the Docker container's own localhost, not the WSL host.

Fix: Use the WSL IP address instead of 127.0.0.1:

```
1 # Get WSL IP
2 hostname -I
3 # Use the returned IP (e.g. 172.22.159.173) in ZAP target
   URL
```

Additional fix for secure app: `app_secure.py` binds to 127.0.0.1 by default. Temporarily change to 0.0.0.0 to allow Docker to reach it:

```
1 sed -i 's/host="127.0.0.1"/host="0.0.0.0"/' part1/app_secure
   .py
2 # Revert after scanning
3 sed -i 's/host="0.0.0.0"/host="127.0.0.1"/' part1/app_secure
   .py
```

5.6.2 Results — Vulnerable App (port 5000)

Finding	Risk	Attack Enabled
Missing Anti-clickjacking Header	Medium	Clickjacking via iframe embedding
X-Content-Type-Options Missing	Low	MIME sniffing attacks
Server Version Leak	Low	Reveals Werkzeug version for targeted exploits
CSP Header Not Set	Medium	No browser-level XSS defense
Permissions Policy Missing	Low	No control over browser features
CORP Header Missing	Low	Cross-origin resource access
Total	6 warnings, 0 failures, 60 passes	

Table 5.8: ZAP baseline scan on vulnerable app

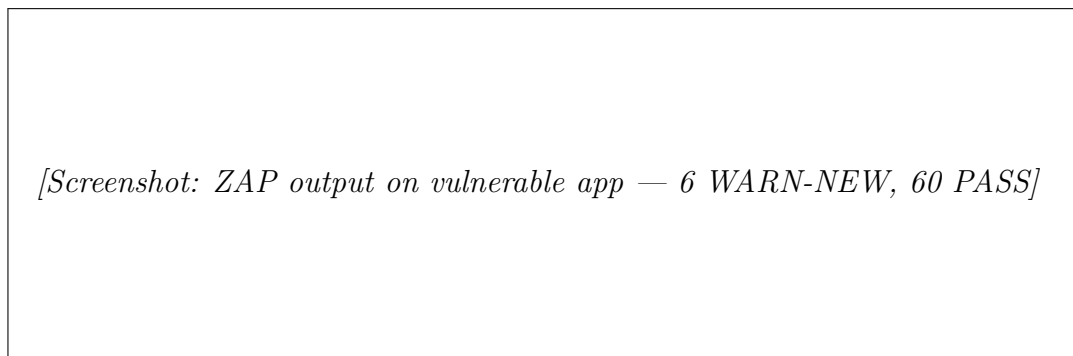
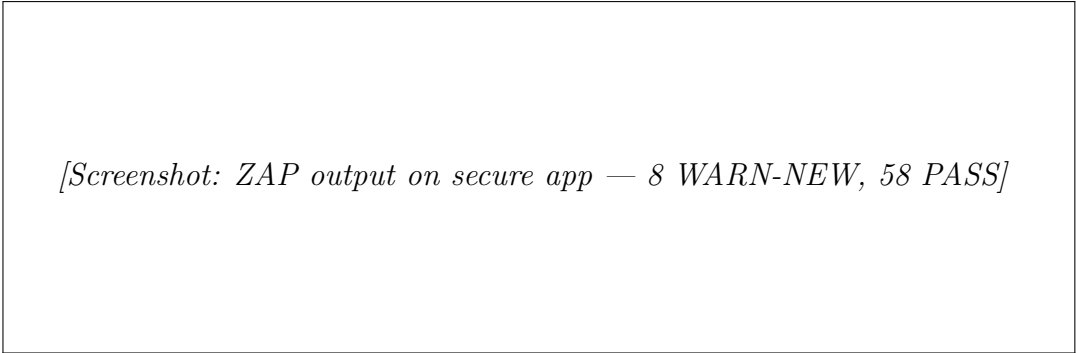


Figure 5.6: OWASP ZAP DAST scan on vulnerable application

5.6.3 Results — Secure App (port 5001)

Finding	Risk	Notes
Missing Anti-clickjacking Header	Medium	Reduced from 4 to 2 instances
X-Content-Type-Options Missing	Low	Reduced from 4 to 2 instances
Server Version Leak	Low	Still present
CSP Header Not Set	Medium	To be fixed in Part 3
Permissions Policy Missing	Low	To be fixed in Part 3
CORP Header Missing	Low	To be fixed in Part 3
Debug Error Messages	Info	New: /login returns 500 (missing templates)
Application Error Disclosure	Info	New: Same cause — environment issue
Total	8 warnings, 0 failures, 58 passes	

Table 5.9: ZAP baseline scan on secure app



[Screenshot: ZAP output on secure app — 8 WARN-NEW, 58 PASS]

Figure 5.7: OWASP ZAP DAST scan on secure application

Why did the secure app get more warnings? The 2 new warnings (Debug Error Messages and Application Error Disclosure) were caused by the `/login` endpoint returning a 500 error due to the missing `login.html` template in the test environment — not a real security issue. In production with templates present, these warnings would disappear. The remaining header-related warnings are addressed in Part 3 via `security_monitoring.py`.

Note on ZAP limitations: The baseline scan (`zap-baseline.py`) performs passive analysis only. It did not detect SQL injection or XSS because those endpoints require authentication or specific parameters. A full active scan (`zap-full-scan.py`) would detect these but takes significantly longer.

5.7 CI/CD Pipeline with GitHub Actions

What is a Security Gate?

A security gate is an automated check in the CI/CD pipeline that blocks a pull request if a security condition is not met. This enforces that insecure code can never reach production.

5.7.1 Pipeline Jobs Overview

Job	Tool	Purpose	Gate
SAST	Bandit	Scan Python code	Block if HIGH severity
SCA	Safety	Check dependencies	Block if critical CVE
Security Tests	pytest	Validate security behavior	Block if tests fail
Secret Scan	Gitleaks	Detect leaked credentials	Block if secret found
Container Scan	Trivy	Scan Docker image	Advisory only
DAST	OWASP ZAP	Dynamic scanning	Advisory only
Security Gate	Aggregator	Final merge check	Block if any job failed

Table 5.10: CI/CD pipeline security gates

5.7.2 Problems Encountered and Fixes

Problem 1 — Invalid workflow syntax: The original devsecops-pipeline.yml used hashFiles() in a job-level if: condition, which is not supported by GitHub Actions:

```
1 # INVALID
2 if: hashFiles('Dockerfile') != ''
3
4 # FIX
5 if: true
```

Problem 2 — Wrong file paths: The original pipeline referenced code/session1/ and code/session2/ paths that do not exist in the cloned repository. All paths were updated to match the actual structure:

```
1 # ORIGINAL (wrong)
2 bandit -r code/session1/app_secure.py
3 safety check -r code/session1/requirements.txt
4 pytest code/session2/test_security.py
5
6 # FIXED
7 bandit -r app.py # scans root app.py
8 safety check -r requirements.txt
9 PYTHONPATH=../part1 pytest part2/test_security.py
```

Problem 3 — Pipeline scanning part1/app_vulnerable.py: Since the vulnerable app exists in part1/, Bandit would always find HIGH severity issues regardless of which branch was being tested.

Fix: The pipeline was configured to scan only app.py at the repository root. Each branch places a different version there:

```
1 # insecure-app branch
2 cp part1/app_vulnerable.py app.py # Pipeline FAILS
3
4 # main branch
5 cp part1/app_secure.py app.py # Pipeline PASSES
```

5.7.3 Pipeline Execution Results

insecure-app branch — pipeline correctly fails:

[Screenshot: GitHub Actions on insecure-app branch — Security Unit Tests failing]

Figure 5.8: Pipeline failing on insecure-app branch

main branch (secure app) — more jobs pass:

[Screenshot: GitHub Actions on main branch — SAST, SCA, Security Tests passing]

Figure 5.9: Pipeline on main branch with secure app

Pull Request — security gate blocking merge:

[Screenshot: PR page showing 7 failing checks blocking merge from insecure-app to main]

Figure 5.10: Security gate blocking merge of insecure branch via Pull Request

Remaining pipeline failures on main branch:

Secret Scanning still fails because the fake AWS credentials in `README.md` are detected by Gitleaks. These are intentional examples from the Gitleaks documentation. Fix: add a `.gitleaks.toml` allowlist file.

Container Security Scan fails because the **Dockerfile** is in **part3/** not at the root. Fix: either move it or update the pipeline path.

These are known infrastructure issues, not security vulnerabilities in the application.

5.7.4 Key Principle — Shift-Left Security

The pipeline implements **shift-left security** — vulnerabilities are caught at commit time, before code reaches staging or production. Each security gate is a hard block: if Bandit finds a HIGH severity issue or Gitleaks finds a secret, the pull request cannot be merged until the issue is resolved. This transforms security from a final gate into a continuous practice embedded in every development cycle.

6. Part 3 — Harden & Monitor

Note: This chapter will be completed after finishing Part 3 of the lab. Sections covered: Docker hardening, WAF, security headers, and incident response.

6.1 Docker Security Hardening

6.2 Runtime Security Monitoring

6.3 Security Headers

6.4 Incident Response Exercise

7. Conclusion

Note: Conclusion to be written after completing all three parts of the lab.

References

1. OWASP Top Ten. *OWASP Foundation*. <https://owasp.org/www-project-top-ten/>
2. STRIDE Threat Modeling. *Microsoft Security*. <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>
3. Bandit Documentation. *PyCQA*. <https://bandit.readthedocs.io/>
4. Safety Documentation. <https://pyup.io/safety/>
5. CIS Docker Benchmark v1.6. *Center for Internet Security*.
6. Werkzeug Documentation — safe_join. <https://werkzeug.palletsprojects.com/>
7. Flask Security Documentation. <https://flask.palletsprojects.com/en/latest/security/>
8. Lab Repository. <https://github.com/said-dev-ops/dev-sec-ops-lab-1>